SSOT — SINGLE SOURCE OF TRUTH

Manual Canônico de Templates

A Gênese, Propósito, Arquitetura e Operação das Peças e Governança do Ecossistema LegoLoop

Análise Detalhada das 6 Dimensões de Cada Template:

Gênese (Por Que Nasceram) • Propósito (Para Que Existem) • Mecanismo (Como Operam)

Momento (Quando Agem) • Topologia (Onde Habitam) • Atores (Por Quem São Usados)

Repositório: LegoLoop Engine

Data: 2026-11-05

Versão do SSOT: v2.0.0 (Isomórfica e Hermética)

Norma de Governança: AGENTS.md (Regras 1 a 8)

Diretório Canônico: `docs/templates/`

Status: Padrão Ouro de Engenharia

Sumário Executivo

1. A Gênese dos Templates — Por Que e Para Que Nasceram? 3

1.1 A Crise Epistemológica da V1 e a Ilusão da "Qualidade Teórica" 3

1.2 A Transição da V1 para a V2: Troca de Promessa por Verificação 3

2. Visão Sistêmica do Ecossistema 5

3. A Cascata Canônica de 4 Estágios (Análise Pormenorizada) 6

3.1 TEMPLATE_INTENT.md — A Declaração de Intenção e "O Porquê" 6

3.2 TEMPLATE_SPEC.md — A Especificação Executável e "O Quê" 8

3.3 TEMPLATE_PLAN.md — O Fatiamento Vertical e "O Como" (Humano) 10

3.4 TEMPLATE_ROUTE.json — A Matriz Isomórfica e "A Máquina" 12

4. A Sextilha Canônica de Peças Reutilizáveis 14

4.1 TEMPLATE_PIECE_SKILL.md — A Peça Cognitiva (Especialista) 14

4.2 TEMPLATE_PIECE_SCRIPT.ps1 — A Peça Mecânica (Transformação Determinística) 15

4.3 TEMPLATE_PIECE_HOOK.json — O Interceptador de Eventos de Harness 17

4.4 TEMPLATE_PIECE_MCP.json — A Ponte de Conexão com Ferramentas Externas 17

4.5 TEMPLATE_PIECE_GATE.json — O Juiz Binário e Portões de Parada (DSL) 19

4.6 TEMPLATE_PIECE_LOOP.json — O Grafo Acíclico Dirigido (DAG Orquestrador) 20

5. A Infraestrutura Go — `templates.go` e `templates_test.go` 21

5.1 Por Que os Templates Foram Embutidos no Binário (`templates.go`)? 21

5.2 Como os Testes Garantem a Imutabilidade (`templates_test.go`) 22

6. Matriz Comparativa das 6 Perguntas Cardeais 23

7. Ciclo de Vida Prático: A Esteira de Fabricação em Ação 25

8. Conclusão e Filosofia de Engenharia 25

1. A Gênese dos Templates — Por Que e Para Que Nasceram?

1.1 A Crise Epistemológica da V1 e a Ilusão da "Qualidade Teórica"

Na versão inicial (V1) do ecossistema, os templates existiam sob rótulos entusiasmados como "Padrão Ouro Universal" e "Contratos Perfeitos". No entanto, uma auditoria rigorosa de código e execução empírica revelou um abismo de engenharia:

- **Ausência de Verificação Mecânica:** Os templates continham dezenas de campos meramente decorativos (como o atributo `commands:` em skills) que nenhum parser, CLI ou motor em Go realmente lia ou validava.
- **Alucinação e Amnésia de Agentes LLM:** Ao receber pedidos abertos de desenvolvimento, cada agente de IA reinventava a estrutura de pastas, suprimia seções vitais, criava funções vazias repletas de stubs (`// TODO: implementar depois`, `pass`, mocks fictícios) e quebrava o determinismo do projeto.
- **Divergência entre Intenção Humana e Execução de Máquina:** O plano de ação em texto corrido não correspondia à rota executável em JSON. Mudava-se um timeout no texto do plano, mas a máquina executava com parâmetros padrão ou desatualizados.
- **Poluição de Contexto (Context Bloat):** Agentes chamados de forma genérica liam centenas de linhas irrelevantes, queimavam cotas de tokens desnecessárias e falhavam por perda de atenção ao misturar o "porquê", o "o quê" e o "como" num único emaranhado textual.

1.2 A Transição da V1 para a V2: Troca de Promessa por Verificação

Os templates canônicos da V2 nasceram em novembro de 2026 com uma premissa fundamental: **se um template não é lido, validado e compilado por um componente real do repositório, ele não é engenharia; é apenas decoração.**

🎯 A Intenção Fundamental de Criação dos Templates V2

A meta foi blindar o ciclo de vida do software contra a entropia dos modelos de linguagem.

Os templates nasceram para:

1. **Fixar o Single Source of Truth (SSOT):** Centralizar em `docs/templates/` a gramática definitiva de todo o repositório, embutida diretamente no binário `legoloop` via `go:embed`.
2. **Impor Isomorfismo 1:1:** Garantir que todo plano aprovado por um humano tenha um espelho exato, auditável e compilável em JSON lido pelo orquestrador de execução.
3. **Instituir a Execução Estéril por Bloco:** Permitir que cada nó do pipeline rode em isolamento hermético (`orchestrix_hermetic`), sem poluição de contexto e sem subagentes parasitas.
4. **Garantir o Padrão Zero Stubs e TDD Rigoroso:** Todo artefato produzido pelos templates deve ser testado, funcional, com critérios binários `AC-*` e livre de placeholders não preenchidos.

2. Visão Sistêmica do Ecossistema

O repositório divide seus templates canônicos em dois pilares complementares: a **Cascata Canônica de 4 Estágios** (que rege o fluxo de ponta a ponta de qualquer demanda) e a **Sextilha Canônica de Peças Reutilizáveis** (os blocos atômicos com os quais qualquer esteira de automação é montada).

A CASCATA CANÔNICA EM 4 ESTÁGIOS

1. INTENT	2. SPEC	3. PLAN	4. ROUTE
O Porquê	*O Quê*	*O Como*	*A Máquina*
Alinhamento socrático, dor real e não-objetivos.	Premissas, fronteiras e critérios BDD `AC-*`.	Fatiamento vertical humano e épicos.	Matriz isomórfica JSON para agentes estéreis.

A SEXTILHA CANÔNICA DE PEÇAS ATÔMICAS

1. SKILL (`.md`)	2. SCRIPT (`.ps1`)	3. HOOK (`.json`)
Habilidade cognitiva de agente com I/O fechado.	Transformação atômica mecânica em PowerShell.	Intercepção nativa de eventos antes de chamadas de ferramenta.

4. MCP (`.json`)	5. GATE (`.json`)	6. LOOP (`.json`)
Conexão com ferramentas externas via Model Context Protocol.	DSL de verificação binária e portões humanos.	Grafo Acíclico Dirigido (DAG) de orquestração.

3. A Cascata Canônica de 4 Estágios (Análise Pormenorizada)

3.1 TEMPLATE_INTENT.md — A Declaração de Intenção e "O Porquê"

💡 1. Por que nasceu e qual foi a intenção de criá-lo?

Historicamente, agentes e desenvolvedores iniciavam o código sem saber exatamente **por que** estavam realizando aquela tarefa. Saltava-se diretamente da ideia para o código, gerando escopos inflados, refatorações clandestinas e perda de foco. O `TEMPLATE_INTENT.md` nasceu para forçar uma **pausa socrática**: antes de tocar em qualquer contrato ou arquivo, deve-se provar a dor real, estabelecer quem sofre com o problema, definir o estado atual versus desejado e, crucialmente, declarar **o que NÃO faremos** (Não-Objetivos).

💡 2. Para que existe?

Existe para alinhar o entendimento humano e o objetivo estratégico do modelo em um documento de baixíssima ambiguidade. Impõe uma pontuação de confiança socrática (mínimo de 95% para autorizar a saída de status `DRAFT` para `UNDER_REVIEW`). Enquanto houver premissas nebulosas, a intenção não avança para especificação técnica.

💡 3. Como é utilizado?

- Preenche-se via entrevista socrática conduzida pelo motor `interview` (`interview.MaterializeIntent`) ou pela skill `skills/aidd-grill`.
- Possui exatamente 6 seções numeradas obrigatórias:
  1. *Contexto e Motivação* (Problema real, evidência, dor, quem sofre, antes vs depois).
  2. *Escopo Inegociável* (Objetivos primários explícitos e lista fechada de proibições).
  3. *Mapeamento Preliminar de Blocos e Subloops* (Tabela relacionando blocos a skills, scripts, subloops e stop gates).
  4. *Governança Operacional e Restrições Inegociáveis* (Gatilho trigger, isolamento, teto de tokens).
  5. *Riscos Mapeados e Planos de Contingência* (Matriz de risco, impacto e ação mitigadora).
  6. *Critérios de Conclusão da Intenção e Stop-Gate 01* (Checklist binário de fechamento).
- **Regra de Fechamento:** Todos os placeholders `{{...}}` devem ser substituídos. O gate é: `!has_pattern('docs/intents/<arquivo>.md', '\{\{')`.

💡 4. Quando e onde é utilizado?

- **Quando:** No exato marco zero de qualquer nova demanda, feature, refatoração ou resolução de bug. Antes de qualquer linha de código ou especificação formal.
- **Onde:** Salvo na pasta `docs/intents/` sob a nomenclatura padronizada ISO: `YYYY-MM-DD_INTENT_<NOME>.md`. Consumido pelo motor `interview` e lido pelo gerador de especificações `specgen`.

💡 5. Por quem é utilizado?

- **Pessoas:** O desenvolvedor, o Product Owner ou arquiteto humano que expressa a necessidade.
- **Agentes e Motores:** Conduzido pelo agente de entrevista socrática (`aidd-grill` ou motor Go `interview`), e consumido posteriormente pelo `specgen`.

3.2 TEMPLATE_SPEC.md — A Especificação Executável e "O Quê"

💡 1. Por que nasceu e qual foi a intenção de criá-lo?

Nasceu para resolver a maior causa de falha em engenharia com IA: **especificações vagas e não-testáveis**. Promessas como "o sistema deve ser rápido" ou "o módulo deve ser seguro" são inúteis para um LLM ou para um teste automatizado. O template nasceu da fusão metodológica de Addy Osmani (premissas técnicas e fronteiras de ação rígidas), Matt Pocock (costuras de teste e isolamento de dependências) e AIDD-Spec (critérios binários BDD `AC-*`).

💡 2. Para que existe?

Existe para ser a fonte de verdade funcional e técnica da solução. Seu objetivo primário é fornecer um conjunto inegociável de Critérios de Aceitação BDD (`AC-XX.Y` e `AC-NF-XX`), incluindo obrigatoriamente cenários de falha e caminhos de erro. Nenhum plano pode ser desenhado sem ter como âncora direta um critério desta especificação.

💡 3. Como é utilizado?

- Alimentado diretamente pela INTENT aprovada. O motor `specgen.Generate` embute este template em seu prompt para estruturar o documento.
- É composto por 6 seções rigorosas:
  1. *Premissas Técnicas Assumidas (Addy Osmani):* Runtime (versão do Go, SO), armazenamento, segurança de rede e pacotes internos reutilizados.
  2. *Fronteiras de Ação & Regras de Governança (Boundaries):* Regras explícitas em três colunas: **Sempre Fazer**, **Perguntar Antes** e **Nunca Fazer** (Zero Stubs, sem segredos em git, sem dependências não autorizadas).
  3. *Visão Geral, Arquitetura & Costuras de Teste (Test Seams — Pocock):* Diagrama Mermaid de arquitetura e declaração cirúrgica de onde o código pode ser interceptado para testes sem mocks fictícios.
  4. *Contratos de Dados, Tipos e Schemas:* Estruturas Go ou JSON completas, com validação de limites.
  5. *Critérios de Aceitação Binários BDD (Gherkin):* Todos os critérios no formato `DADO... QUANDO... ENTÃO...`. Todo épico deve possuir no mínimo um critério de caminho feliz e um critério de caminho de falha.
  6. *Critérios Não Funcionais e Stop-Gate 02:* Desempenho, isolamento e aprovação humana do escopo.

- **Gate Mecânico:** Deve passar nas funções DSL `has_binary_criteria('SPEC.md') && has_acceptance_criteria('SPEC.md')`.

💡 4. Quando e onde é utilizado?

- **Quando:** Imediatamente após a aprovação humana da INTENT (Stop-Gate 01). Precede a criação do plano e da rota de execução.
- **Onde:** Persistido em `docs/specs/YYYY-MM-DD_SPEC_<NOME>.md`. Consumido pelo motor `plangen` para desenhar o plano de tickets.

💡 5. Por quem é utilizado?

- **Pessoas:** Validado e revisado pelo desenvolvedor humano no Stop-Gate 02.
- **Agentes e Motores:** Gerado pelo agente de especificação ou motor `specgen`, consumido pelo `plangen` e auditado pelo motor de reputação.

3.3 TEMPLATE_PLAN.md — O Fatiamento Vertical e "O Como" (Humano)

💡 1. Por que nasceu e qual foi a intenção de criá-lo?

Muitas vezes projetos de IA geram "planos gigantescos" de 50 passos sequenciais que perdem o controle no meio do caminho, ou fatiam o sistema em camadas horizontais inoperantes (ex: criar todo o banco de dados sem nenhuma tela, depois todo o backend sem validação). O `TEMPLATE_PLAN.md` nasceu para obrigar o fatiamento em **fatias verticais (tracer bullets)**: cada ticket entrega valor de ponta a ponta e fecha um critério específico `AC-*` da SPEC. Além disso, nasceu para ser o **espelho legível por humanos** do arquivo de rota da máquina.

💡 2. Para que existe?

Existe para permitir que o engenheiro humano compreenda, audite e aprove a estratégia de execução do pipeline: níveis de supervisão (`nano`, `micro`, `epic`, `autonomous`), limite de paralelismo, teto de tokens por bloco, timeouts e estratégia de rollback caso um teste falhe.

💡 3. Como é utilizado?

- O documento organiza os tickets agrupados em Épicos funcionais. Cada ticket descreve:
  - **Identificador Canônico:** Padrão `TICK-NN.M`.
  - **Natureza:** `nano` (ajustes mínimos), `micro` (unidade com TDD), `macro` (integração de subsistema).
  - **Modo de Execução:** `tdd_red_green`, `sequential` ou `parallel`.
  - **Isolamento e Roteamento Cognitivo:** Proibição expressa de subagentes (`subagents_allowed: false`), definição de modelo primário (ex: Claude Sonnet via 9Router) e modelo de contingência/fallback.
  - **Contratos de Tooling e I/O:** Arquivos que podem ser tocados (`source_files`, `test_files`). O agente é impedido de modificar arquivos fora dessa lista fechada.
  - **Comando de Verificação e Gate DSL:** Script ou comando que atesta o sucesso com `exit_code == 0`.
  - **Rastreabilidade Estrita:** Referência obrigatória ao critério de aceite correspondente (`Spec Ref: AC-XX.Y`).

💡 4. Quando e onde é utilizado?

- **Quando:** Gerado na Fase 3, logo após a homologação da SPEC. Deve ser aprovado pelo usuário no Stop-Gate 03 antes de disparar o código.

- **Onde:** Salvo em `docs/plans/YYYY-MM-DD_PLAN_<NOME>.md`. Espelhado 1:1 com `docs/routes/YYYY-MM-DD_ROUTE_<NOME>.json`.

💡 5. Por quem é utilizado?

- **Pessoas:** Lido, auditado e assinado pelo desenvolvedor humano.
- **Agentes e Motores:** Gerado pelo motor `plangen.Generate` em consonância perfeita com o `TEMPLATE_ROUTE.json`.

3.4 TEMPLATE_ROUTE.json — A Matriz Isomórfica e "A Máquina"

💡 1. Por que nasceu e qual foi a intenção de criá-lo?

Nasceu para resolver a ambiguidade da linguagem natural na automação. Agentes de execução não devem interpretar texto livre para decidir quais comandos rodar, que arquivos podem tocar ou que gates devem ser satisfeitos. A intenção de criar o `TEMPLATE_ROUTE.json` foi construir uma **matriz de nós estritamente estruturada e tipada em JSON**, onde cada nó é um contrato computacional fechado e imutável.

💡 2. Para que existe?

Existe como o orquestrador executável lido pela CLI do LegoLoop (`legoloop run`) e pelo executor de blocos `orchestrix_hermetic`. Fornece contingência dupla automática (harness primário vs fallback, modelo primário vs fallback), garantia de isolamento hermético e gestão de defeitos em tempo real.

💡 3. Como é utilizado?

- Consiste num JSON validado por schema (`https://legoloop.dev/schemas/route.v2.json`) contendo a lista de `nodes`.
- Cada nó possui uma estrutura isomórfica rigorosa testada em `templates.RouteKeys()`:
  - `id`, `label`, `nature`, `type` (`unit_task`, `subloop`, `stop_gate`), `ref`, `status`.
  - `isolation`: `workspace` (`local`, `git_worktree`, `tempdir`), `executor` (`orchestrix_hermetic`), `subagents_allowed` (sempre `false`).
  - `timeout_sec` e `token_budget` (`max_output_tokens` máximo de 600 tokens para impedir context bloat).
  - `depends_on`: Grafo explícito de dependências sem ciclos.
  - `routing`: Seleção cirúrgica de modelo (ex: Sonnet para raciocínio complexo, Flash para tarefas mecânicas) com fallback.
  - `tooling`: Ambiente de execução restrito e MCPs permitidos.
  - `inputs` e `output`: Arquivos de entrada, arquivos alvo fechados e `output.gate` (expressão DSL compilada).
  - `output.verify`: Array de comandos argv (ex: `[["go", "test", "-run", "TestAuth", "./..."]]`) **sem passar por interpretador de shell**, eliminando riscos de injeção.

💡 4. Quando e onde é utilizado?

- **Quando:** Gerado simultaneamente com o PLAN.md pelo motor `plangen`. É consumido durante a execução da esteira pelo orquestrador.
- **Onde:** Salvo em `docs/routes/YYYY-MM-DD_ROUTE_<NOME>.json`. Consumido por `cmd/legoloop`, `internal/orchestrator` e pelo Studio.

💡 5. Por quem é utilizado?

- **Pessoas:** Raramente editado manualmente; inspecionado via LegoLoop Studio.
- **Agentes e Motores:** Consumido mecanicamente pelo executor de esteira (`legoloop run`), atualizado dinamicamente nos campos `status` e `defects[]`.

4. A Sextilha Canônica de Peças Reutilizáveis

Enquanto a cascata governa o ciclo de vida do projeto, a sextilha de peças padroniza as **unidades de trabalho e componentes plugáveis** que operam dentro do LegoLoop.

4.1 TEMPLATE_PIECE_SKILL.md — A Peça Cognitiva (Especialista)

💡 1. Por que nasceu e qual foi a intenção de criá-lo?

Na V1, as skills eram documentos desordenados, com textos vagos, sem escopo definido de escrita e com campos fictícios (como `commands:` que nenhum parser consumia). Os agentes alucinavam permissões de escrita em todo o disco. O `TEMPLATE_PIECE_SKILL.md` foi criado para transformar a skill em um **componente com contrato estrito de entrada e saída (I/O)**, com escopo cirúrgico de escrita (`target_files`) e regras negativas inegociáveis.

💡 2. Para que existe?

Existe para instruir um modelo de linguagem sobre como executar uma especialidade única (ex: modelar BDD, refatorar código, gerar documentação), limitando seu contexto aos arquivos necessários e estabelecendo regras de ouro sobre o que NÃO fazer.

💡 3. Como é utilizado?

- Contém um frontmatter YAML lido pelo scanner `internal/assets/skills.go` e pelo grafo semântico `internal/graph`:
  - `name`: Identificador idêntico ao nome da pasta (ex: `skills/minha-skill`).
  - `description`: Máximo de 300 caracteres, iniciando com gatilho de uso (“Use quando...”) e proibição de uso (“NÃO use quando...”).
  - `consumes` e `produces`: Tipos formais do grafo (ex: `Artifact:Spec` → `Artifact:Code`).
  - `artifact` e `gate`: Nome do artefato mecânico esperado e validação DSL de aceite.
  - `suggested_model`: Modelo mais econômico e eficiente para aquela tarefa.
- O corpo do Markdown estabelece: Casos de uso concretos, Target Files fechados, protocolo operacional e regras negativas.

💡 4. Quando, Onde e Por Quem?

- **Quando:** Invocada quando um nó do tipo `skill` é acionado pelo orquestrador.

- **Onde:** Localizada em `skills/<nome-da-skill>/SKILL.md`. Descoberta automaticamente pelo scanner de assets do LegoLoop.
- **Por quem:** Lida pelo modelo alocado ao nó em contexto estéril; criada e evoluída por desenvolvedores ou pelo comando `legoloop upgrade`.

4.2 TEMPLATE_PIECE_SCRIPT.ps1 — A Peça Mecânica (Transformação Determinística)

💡 **1. Por que nasceu e qual foi a intenção de criá-lo?**

Modelos de linguagem são péssimos e caros para tarefas mecânicas repetitivas (como normalização de quebras de linha, parsing de regex em milhares de arquivos, formatação de dados). Além disso, scripts improvisados em pipelines frequentemente quebram silenciosamente, deixam arquivos temporários órfãos e não tratam erros. Este template nasceu para fornecer um **padrão determinístico de script PowerShell 5.1+ corporativo, idempotente, atômico e confinado à raiz**.

💡 **2. Para que existe?**

Existe para garantir que qualquer automação procedural no ecossistema rode com segurança máxima: suporte a `-DryRun`, escrita via arquivo temporário com substituição atômica, códigos de saída padronizados e codificação UTF-8 com BOM para compatibilidade universal no Windows.

💡 **3. Como é utilizado?**

- Segue rigorosamente a convenção tripartite de exit codes da CLI LegoLoop:
  - `0`: Sucesso total (inclui "nada a fazer" e `--DryRun`).
  - `1`: Falha de execução (erro de E/S, permissão, disco).
  - `2`: Entrada inválida (arquivo alvo ausente ou tentativa de escape fora da raiz do projeto).
- É testado mecanicamente em `templates_test.go` sob 7 cenários hostis (alvo inexistente, fora da raiz, raiz-irmã, arquivo somente leitura).

💡 **4. Quando, Onde e Por Quem?**

- **Quando:** Executado em nós mecânicos de transformação, validação de segurança e linters.
- **Onde:** Armazenado em `scripts/`, `tools/`, `.aidd/scripts/` ou `.claude/scripts/`.

- **Por quem:** Disparado pela CLI LegoLoop, por hooks de harness ou por desenvolvedores no terminal.

4.3 TEMPLATE_PIECE_HOOK.json — O Interceptador de Eventos de Harness

💡 1. Por que e Para Que Nasceu?

Nasceu da necessidade de governança de segurança em tempo de execução. Agentes com acesso a ferramentas de escrita e shell podem inadvertidamente gravar credenciais, apagar arquivos críticos ou executar comandos destrutivos. O template de hook nasceu para fornecer uma **camada de interceptação preventiva antes que qualquer ferramenta do harness seja acionada**.

💡 2. Como é utilizado?

- Formato JSON nativo consumido pelo Claude Code (`settings.json`).
- Intercepta eventos como `PreToolUse` com regex de matcher estreito (ex: `"Write|Edit|MultiEdit"`).
- O script acionado lê o payload do evento em JSON via `stdin`. Se o script retornar `exit 0`, a ferramenta é liberada; se retornar `exit 2`, a ação é abortada e o erro é devolvido ao modelo para autocorreção.
- Proíbe estritamente a gravação de segredos literais nos comandos (detectado por `assets.ContainsSecret`).

💡 3. Quando, Onde e Por Quem?

- **Quando:** Em tempo real, a cada intenção do agente de editar arquivos ou executar comandos.
- **Onde:** Configurado em `.claude/settings.json` ou nas configurações do harness em uso.
- **Por quem:** Executado pelo motor do harness (ex: Claude Code) acionando scripts de defesa como `guard-secrets.ps1`.

4.4 TEMPLATE_PIECE_MCP.json — A Ponte de Conexão com Ferramentas Externas

💡 1. Por que e Para Que Nasceu?

O Model Context Protocol (MCP) tornou-se o padrão da indústria para conectar IAs a bancos de dados, navegadores e serviços. No entanto, configurações despadronizadas causavam vazamento de chaves de API e falhas de compatibilidade entre sistemas operacionais (notadamente o Windows lidando com comandos Node/npx). Este template nasceu para **padronizar o registro de servidores MCP seguros e universais**.

💡 2. Como é utilizado?

- Formato de trecho de `.mcp.json` lido por `internal/assets/mcps.go`.

- Impõe que segredos nunca sejam colocados em texto puro: utiliza interpolação de variáveis de ambiente `${VAR}`.
- No Windows, padroniza a execução de executáveis Node/npx através do envelopamento seguro `cmd /c npx`.
- O contrato de ferramentas deve ser estrito com `inputSchema` fechado (`additionalProperties: false`).

💡 3. Quando, Onde e Por Quem?

- **Quando:** No carregamento inicial do ambiente de desenvolvimento ou ao conectar capacidades externas (ex: REA, Playwright).
- **Onde:** No arquivo `.mcp.json` na raiz do projeto.
- **Por quem:** Lido pelo cliente MCP do harness e monitorado pelo scanner de integridade do LegoLoop.

4.5 TEMPLATE_PIECE_GATE.json — O Juiz Binário e Portões de Parada (DSL)

💡 1. Por que e Para Que Nasceu?

Antes da existência de uma DSL formal de gates, a validação de etapas dependia do próprio modelo dizer se seu trabalho estava bom — um caso clássico de "a raposa cuidando do galinheiro". Os LLMs frequentemente declaravam tarefas prontas mesmo com testes falhando ou arquivos incompletos. O `TEMPLATE_PIECE_GATE.json` nasceu para instituir um **árbitro determinístico mecânico**, onde o sucesso de uma etapa depende de funções booleanas matemáticas puras e verificação de exit codes reais.

💡 2. Para que existe e Como é utilizado?

- Fornece um catálogo de receitas de gates compiláveis pelo motor `internal/gate`:
  1. **Gate Mecânico:** Avalia a saída de comandos de teste (`exit_code == 0` sobre `go test ./...`).
  2. **Gate de Qualidade de Artefato:** Verifica existência física, tamanho e ausência de stubs:

     ```
     file_exists('ARTEFATO') && word_count('ARTEFATO') > 50 && !has_pattern('ARTEFATO', '(?i)\\b(TODO|FIXME)\\b').
     ```

  3. **Gate de Contrato de Spec:** Exige checklist binário e seções BDD:

     ```
     has_binary_criteria('SPEC') && has_acceptance_criteria('SPEC').
     ```

  4. **Gate de Schema JSON:** Valida o documento contra um JSON Schema formal: `schema_valid('SCHEMA', 'JSON')`.
  5. **Stop-Gate Humano:** Pausa incondicionalmente a esteira e solicita autorização explícita do desenvolvedor:

     ```
     prompt('Pergunta ao humano').
     ```

- **Regra Inviolável (AGENTS.md):** Stop-gates humanos não podem ser simulados ou aprovados por IA.

💡 3. Quando, Onde e Por Quem?

- **Quando:** Avaliado no encerramento de cada nó do DAG ou ao final de cada ticket da ROUTE.
- **Onde:** Embutido no campo `output.gate` dos nós ou como arquivo de receitas em `docs/templates/`.
- **Por quem:** Compilado e executado pelo motor Go `internal/gate` (`gate.Compile`), reportando ao orquestrador.

4.6 TEMPLATE_PIECE_LOOP.json — O Grafo Acíclico Dirigido (DAG Orquestrador)

💡 1. Por que e Para Que Nasceu?

Processos complexos de engenharia de software não são lineares: possuem ramificações, dependências cruzadas e nós que podem rodar em paralelo. Tentar orquestrar isso via prompts encadeados causa perda de estado e caos operacional. O template de Loop nasceu para trazer a **teoria de grafos (DAG - Directed Acyclic Graph) para o coração da engenharia com agentes**.

💡 2. Como é utilizado?

- Estrutura JSON canônica validada por `internal/dag` (`dag.Parse`) e armazenada por `internal/loop` (`loop.Store`).
- Define nós com identificadores únicos, tipo da peça (`skill`, `script`, `subloop`), referências relativas, dependências explícitas (`depends_on`), alocação de modelo cognitivo e contrato de saída com artefato e gate.
- É validado pelo comando `legoloop validate <arquivo.json>`, garantindo que não existem ciclos infinitos nem referências a nós inexistentes.

💡 3. Quando, Onde e Por Quem?

- **Quando:** Utilizado para definir loops completos de trabalho (ex: Loop de Fabricação, Loop de Auditoria 4F, Loop de Upgrade de Peças).
- **Onde:** Armazenado na pasta de loops do ecossistema e embutido no motor da CLI.
- **Por quem:** Lido e executado pelo motor de execução de DAG do LegoLoop.

5. A Infraestrutura Go — `templates.go` e `templates_test.go`

A grande inovação que distingue a V2 do LegoLoop de projetos comuns de automação é a sua **materialização em código compilado**.

5.1 Por Que os Templates Foram Embutidos no Binário (`templates.go`)?

Em sistemas tradicionais, templates ficam como arquivos avulsos no disco. Isso gera três problemas graves:

1. **Risco de Dessincronização:** Uma atualização na CLI ou nos motores não reflete nos templates, ou um desenvolvedor edita um arquivo local e o sistema passa a gerar código inválido.
2. **Duplicação de Strings (DRY Violation):** Os motores em Go (`interview`, `specgen`, `plangen`) precisariam ter cópias embutidas de strings, gerando divergência inevitável entre a documentação e o código.
3. **Fragilidade em Ambientes Instalados:** Ao rodar o binário `legoloop` globalmente via `go install`, os arquivos da pasta `docs/templates/` não estariam acessíveis caso dependessem de leitura de caminho relativo no disco.

A Solução Canônica: O arquivo `docs/templates/templates.go` utiliza a diretiva nativa do compilador Go `//go:embed`:

```
//go:embed TEMPLATE_INTENT.md
var intentMD string

//go:embed TEMPLATE_SPEC.md
var specMD string

//go:embed TEMPLATE_PLAN.md
var planMD string

//go:embed TEMPLATE_ROUTE.json
var routeJSON []byte
```

Dessa forma, os templates em Markdown e JSON na pasta `docs/templates/` são a **Single Source of Truth absoluta**. Ao compilar o executável, eles são fundidos no binário. Os motores de geração leem as

funções `templates.Intent()`, `templates.Spec()`, `templates.Plan()` e `templates.Route()`. Se um arquivo do disco for alterado, os testes unitários acusam a discrepância imediatamente.

5.2 Como os Testes Garantem a Imutabilidade (`templates_test.go`)

O arquivo de teste não apenas verifica texto; ele executa validações funcionais reais usando os módulos de produção do sistema:

- **Validação de Sintaxe JSON:** Garante que todos os 5 templates JSON do diretório são documentos JSON perfeitamente parseáveis.
- **Isomorfismo de Nós:** Valida que todos os nós do `TEMPLATE_ROUTE.json` possuem exatamente as mesmas chaves (`RouteShape`), impedindo nós estruturalmente heterogêneos.
- **Compilação Integral de Gates:** Extrai todos os 17 gates presentes nos templates (nos nós da ROUTE, no PLAN, nas receitas de GATE e no DAG de LOOP) e os compila através do compilador real `gate.Compile`. Se houver uma única função inexistente ou parêntese incorreto, a suíte de testes do repositório quebra.
- **Validação de DAG:** O `TEMPLATE_PIECE_LOOP.json` é injetado no parser `dag.Parse` e armazenado em `loop.Store`, garantindo compatibilidade estrita.
- **Descoberta pelo Scanner de Assets:** O módulo `assets.Discover` varre os templates e comprova que consegue indexar a skill, o hook, o script, o MCP e o loop.
- **Teste do Script PowerShell:** O script `TEMPLATE_PIECE_SCRIPT.ps1` é executado mecanicamente em ambiente de teste contra 7 cenários adversos, confirmando a obediência aos códigos de saída `0`, `1` e `2`.

6. Matriz Comparativa das 6 Perguntas Cardeais

Abaixo, a tabela-mestra consolidada respondendo de forma sintética e conclusiva a todas as dimensões de cada template do ecossistema:

Template	Por Que Nasceu? (Gênese)	Para Que Existe? (Propósito)	Como é Utilizado?	Quando e Onde?	Por Quem?
`TEMPLATE_INTENT.md`	Impedir código prematuro sem diagnóstico da dor real e escopo sem limites.	Alinhar a intenção do usuário via entrevista socrática com confiança $\geq 95\%$.	6 seções; declara problema, dor, quem sofre, escopo e não-objetivos.	Marco zero de qualquer demanda; salvo em `docs/intents/`.	Desenvolvedor humano e motor de entrevista `interview.MaterializeIntent`.
`TEMPLATE_SPEC.md`	Eliminar especificações vagas e não-testáveis que causam alucinação de IA.	Fixar premissas, fronteiras e critérios BDD binários inegociáveis (`AC-*`).	Fusão Osmani + Pocock; tabela de fronteiras e critérios Given-When-Then.	Fase 2 (após Intent aprovada); salvo em `docs/specs/`.	Motor `specgen.Generate` e revisado pelo humano no Stop-Gate 02.
`TEMPLATE_PLAN.md`	Evitar planos horizontais inoperantes e manter o humano no controle da esteira.	Apresentar fatiamento vertical legível de tickets rastreáveis à SPEC.	Espelho 1:1 do ROUTE; define épicos, naturezas, isolamento e stop-gates.	Fase 3 (após Spec aprovada); salvo em `docs/plans/`.	Gerado por `plangen.Generate` e assinado pelo usuário no Stop-Gate 03.
`TEMPLATE_ROUTE.json`	Substituir instruções em texto livre por matriz rígida executável de máquina.	Orquestrar execução de agentes estéreis com contingência dupla e isolamento.	Grafo JSON de nós com inputs, outputs fechados, comandos argv e gates DSL.	Fase de Execução; salvo em `docs/routes/`. Lido por `legoloop run`.	Executor `orchestrix_hermetic`, CLI LegoLoop e motor de rota.
`TEMPLATE_PIECE_SKILL.md`	Superar skills anêmicas e campos falsos da V1 sem contratos de escrita.	Instruir modelo em especialidade atômica com target files fechados.	Frontmatter YAML lido por `assets`; contratos de I/O e regras negativas.	Execução de tarefas cognitivas; salvo em `skills/<nome>/SKILL.md`.	Agentes em contexto estéril e pelo comando `legoloop upgrade`.
`TEMPLATE_PIECE_SCRIPT.ps1`	Eliminar scripts frágeis que deixam lixo temporário e ignoram erros.	Executar transformações mecânicas atômicas, idempotentes e confinadas.	PowerShell 5.1+ com UTF-8 BOM, `-DryRun` e exit codes padronizados (0, 1, 2).	Automações procedurais; salvo em `scripts/`, `tools/`	CLI LegoLoop, hooks do harness e desenvolvedores no terminal.

				ou `.claude/scripts/`.	
`TEMPLATE_PIECE_HOOK.json`	Prevenir que agentes executem ações destrutivas ou gravem segredos.	Interceptar eventos de ferramentas antes da execução e barrar violações.	JSON do Claude Code em `PreToolUse`; script em stdin; exit 2 barra ação.	Tempo real em cada chamada de ferramenta; salvo em `.claude/settings.json`.	Harness Claude Code acionando scripts de defesa em background.
`TEMPLATE_PIECE_MCP.json`	Acabar com configurações de MCP inseguras e falhas de runtime no Windows.	Conectar ferramentas externas e bancos com injeção segura de segredos.	Mapeia servidores com `${VAR}` de ambiente e envelopamento `cmd /c npx`.	Setup de ambiente; configurado na raiz em `.mcp.json`.	Clientes MCP do harness e scanner de assets `internal/assets/mcps.go`.
`TEMPLATE_PIECE_GATE.json`	Impedir que o modelo aprove o próprio trabalho sem comprovação mecânica.	Fornecer expressões booleanas matemáticas puras e portões de parada.	DSL com `exit_code`, `file_exists`, `word_count`, `has_pattern` e `prompt` humano.	Ao final de nós e tickets; embutido em `output.gate` de DAGs e rotas.	Compilador `internal/gate` (`gate.Compile`) e desenvolvedor humano.
`TEMPLATE_PIECE_LOOP.json`	Superar prompts encadeados lineares através da teoria de grafos (DAG).	Definir pipelines complexos com dependências, timeouts e modelos adequados.	JSON validado por `dag.Parse`; nós com dependências acíclicas e gates.	Definição de fluxos complexos reutilizáveis; salvo em diretórios de loop.	Motor de DAG do LegoLoop (`loop.Store`) e comando `legoloop run`.

7. Ciclo de Vida Prático: A Esteira de Fabricação em Ação

Para compreender como esses 10 templates operam em harmonia na vida real, examinemos o fluxo canônico de atendimento de uma demanda no LegoLoop:

FLUXO CRONOLÓGICO DA ESTEIRA LEGOLOOP

Fase 1: Intenção (`TEMPLATE_INTENT.md`)

Humano e agente conversam via entrevista socrática. A dor é dissecada. Escopo fechado. Não-objetivos declarados.

Stop-Gate 01: Confiança $\geq 95\%$. Usuário aprova a Intent.

Fase 2: Especificação Técnica (`TEMPLATE_SPEC.md`)

O motor `specgen` consome a Intent e o código existente. Gera premissas, boundaries e critérios BDD `AC-*`.

Stop-Gate 02: Gate mecânico `has_binary_criteria` + Aprovação humana da arquitetura.

Fase 3: Planejamento e Matriz Isomórfica (`TEMPLATE_PLAN.md` & `TEMPLATE_ROUTE.json`)

O motor `plangen` projeta o fatiamento em tickets verticais. Cria o Markdown para humanos e o JSON para a máquina.

Stop-Gate 03: Aprovação humana do plano de voo e dos tetos de custo/tokens.

Fase 4: Execução Hermética e TDD (`legoloop run`)

A máquina despacha cada nó da ROUTE para um executor isolado (`orchestrix_hermetic`). Agentes usam as **Skills** necessárias, scripts executam transformações (**Script**), gates barram regressões (**Gate**), hooks protegem segredos (**Hook**).

Stop-Gate Final: Suíte de testes `go test` passando + Recompilação fresca do binário.

8. Conclusão e Filosofia de Engenharia

Os templates canônicos do LegoLoop não são meros formulários de preenchimento; são **estruturas de contenção de entropia**. Ao transformar premissas humanas em contratos formais e espelhar o racio-

cínio em especificações de máquina testáveis, o ecossistema atinge o mais alto nível de confiabilidade na engenharia assistida por inteligência artificial.

Nenhuma inteligência artificial é autorizada a pular os stop-gates humanos, nenhum código de produção é aceito com stubs ou sem testes unitários prévios, e nenhum plano é executado sem validação prévia de seus gates matemáticos. Essa é a essência do LegoLoop: **autonomia guiada por rigor determinístico**.